



جامعة صبراتة

كلية تقنية المعلومات الجميل

قسم : هندسة البرمجيات

مادة : انماط تصميم البرمجيات

" نظام إدارة المكتبة باستخدام أنماط التصميم "

للتالبات :

خلود الطاهر عبيد .

رواسي رمضان المرناقي .

تقرير مشروع نظام إدارة المكتبة :

• المقدمة :

يهدف مشروع "نظام إدارة المكتبة" إلى تطوير نظام برمجي متكامل لإدارة المكتبات باستخدام لغة البرمجة Java. تم تصميم النظام ليكون نموذجاً عملياً لتطبيق الأنماط التصميمية (Design Patterns) في بيئة حقيقية.

يعالج النظام مشاكل إدارة الكتب التقليدية من خلال:

- تنظيم الكتب في أقسام هرمية
 - تسهيل عمليات الاستعارة والإرجاع
 - توفير واجهة مستخدم بسيطة وفعالة
 - فصل المسؤوليات بين مكونات النظام
- تم تطوير النظام لتلبية احتياجات المكتبات الصغيرة والمتوسطة، مع إمكانية التوسع مستقبلاً.

النظام يدعم نوعين من المستخدمين:

- أمين المكتبة:
- صلاحيات كاملة لإدارة الكتب والأقسام
- القدرة على إضافة وحذف العناصر
- عرض الإحصائيات والتقارير
- البحث المتقدم في المحتوى
- المستخدم العادي:
- استعارة الكتب المتاحة
- إرجاع الكتب المستعارة
- البحث في الفهرس
- عرض الكتب المتاحة

• مكونات النظام الأساسية:

- واجهة المستخدم (Main.java)
- مدير النظام (LibraryManager)
- هياكل البيانات (Book, Section)

- نظام الأوامر (Command Pattern)

• الأنماط التصميمية المستخدمة :

تم استخدام ثلاثة أنماط تصميم رئيسية:

1 Singleton Pattern (النمط الفردي)

- الغرض: ضمان وجود نسخة واحدة فقط من مدير المكتبة

- الملف: `LibraryManager.java`

- الاستخدام: إدارة الحالة المركزية للنظام

2 Composite Pattern (النمط المركب)

- الغرض: تمثيل الهيكل الهرمي للمكتبة

- الملفات: `LibraryItem.java`, `Book.java`, `Section.java`

- الاستخدام: تنظيم الكتب في أقسام وأقسام فرعية

3 Command Pattern (نمط الأمر)

- الغرض: تغليف العمليات في أوامر منفصلة

- الملفات: `Command.java` والأوامر المختلفة

- الاستخدام: تنفيذ عمليات المكتبة

• تفاصيل نمط Singleton :

التنفيذ في `LibraryManager.java`:

```
} public class LibraryManager
```

```
{private static LibraryManager instance
```

• فوائد استخدام Singleton:

1. نقطة وصول واحدة لجميع بيانات المكتبة.

2. منع تضارب البيانات

3. توفير في استخدام الذاكرة

4. سهولة إدارة الحالة العامة

• تفاصيل نمط Composite :

• الهيكل الأساسي:

// الواجهة المشتركة :

```
}public interface LibraryItem
```

```
{!(void display(int indent
```

// الورقة (Leaf) - الكتاب

```
}public class Book implements LibraryItem
```

```
{{ ... } (public void display(int indent
```

// المركبة (Composite) - القسم

```
}public class Section implements LibraryItem
```

```
!private List<LibraryItem> children
```

```
{{ ... } (public void display(int indent
```

• فوائد استخدام Composite:

1. تمثيل طبيعي للهيكل الهرمي

2. معاملة موحدة للكتب والأقسام

3. سهولة إضافة أنواع جديدة

4. مرونة في عرض البيانات

• تفاصيل نمط Command :

• الهيكل الأساسي:

// واجهة الأمر

```
}public interface Command
```

```
{ !()boolean execute
```

// أمر ملموس - إضافة كتاب

```
}public class AddBookCommand implements Command
```

```
} !()public boolean execute
```

// تنفيذ عملية الإضافة}}

// المنفذ (Invoker)

```
}public class Invoker
```

```
{private Stack<Command> history
```

```
{ { ... } public void executeCommand(Command cmd
```

- فوائد استخدام Command:

1. فصل واجهة المستخدم عن المنطق

2. إمكانية التراجع عن العمليات

3. تسجيل تاريخ العمليات

4. سهولة إضافة عمليات جديدة

- تنفيذ النظام :

- تدفق البيانات في النظام:

1. بدء التشغيل:

- إنشاء نسخة وحيدة من LibraryManager

- تحميل البيانات الافتراضية

- عرض القائمة الرئيسية

2. عملية استعارة كتاب:

- المستخدم ← Main.java (اختيار الاستعارة)

- Main.java ← إنشاء BorrowBookCommand

- Invoker ← تنفيذ الأمر

- LibraryManager ← تحديث حالة الكتاب

- عرض النتيجة للمستخدم

3. عملية إضافة كتاب:

- أمين المكتبة ← إدخال بيانات الكتاب

- التحقق من صحة البيانات

- إنشاء AddBookCommand

- إضافة الكتاب إلى القسم المناسب

- تحديث الفهرس

• هياكل البيانات :

الهياكل المستخدمة:

1. <HashMap<String, Book> :

- لتخزين الكتب باستخدام المعرف (ID)
- وصول سريع ($O(1)$) للكتب
- منع التكرار بواسطة المفتاح

2. <List<LibraryItem> :

- تخزين أبناء الأقسام
- مرونة في الإضافة والحذف
- دعم التكرار بسهولة

3. <Stack<Command> :

- حفظ تاريخ الأوامر المنفذة
- إمكانية التراجع عن العمليات
- تتبع نشاط النظام

4. الهيكل المركب:

- تمثيل علاقات الكتب والأقسام
- دعم التسلسل الهرمي
- سهولة التنقل والعرض

• المميزات الوظيفية :

■ لأمين المكتبة:

- إضافة كتب جديدة مع التحقق من التكرار
- حذف كتب مع تأكيد العملية
- إنشاء أقسام جديدة وتنظيمها
- عرض فهرس كامل للمكتبة
- البحث المتقدم بمعايير متنوعة
- عرض إحصائيات مفصلة

- إدارة عمليات الاستعارة والإرجاع

▪ للمستخدم العادي:

- استعارة الكتب المتاحة

- إرجاع الكتب المستعارة

- البحث في الفهرس

- عرض الكتب المتاحة فقط

- مشاهدة إحصائيات المكتبة

- واجهة مستخدم بسيطة وسهلة

• التحديات والحلول :

▪ التحدي1: إدارة الحالة المركزية

الحل: استخدام Singleton Pattern

النتيجة: ضمان اتساق البيانات وتجنب التضارب

▪ التحدي2: الهيكل الهرمي المعقد

الحل: استخدام Composite Pattern

النتيجة: تمثيل مرن وسهل للمكتبة

▪ التحدي3: فصل واجهة المستخدم

الحل: استخدام Command Pattern

النتيجة: كود نظيف وقابل للصيانة

▪ التحدي4: تتبع العمليات

الحل: Stack في Invoker

النتيجة: حفظ تاريخ كامل للعمليات

▪ التحدي5: البحث في الهيكل المركب

الحل: خوارزمية بحث متكررة

النتيجة: بحث شامل في كل المستويات

• الخاتمة والتوصيات :

• الخاتمة:

نجاح المشروع في تحقيق الأهداف المرجوة من خلال:

1. تطبيق عملي للأنماط التصميمية الثلاثة
2. بناء نظام متكامل لإدارة المكتبات
3. فصل واضح للمسؤوليات بين المكونات
4. واجهة مستخدم سهلة وفعالة

• الدروس المستفادة:

1. أهمية التخطيط المسبق للأنماط
2. فوائد فصل المسؤوليات في الكود
3. مرونة الأنماط التصميمية في حل المشاكل
4. أهمية التوثيق الجيد للكود

• التوصيات المستقبلية:

1. إضافة قاعدة بيانات دائمة
2. تطوير واجهة رسومية
3. دعم استعارة متعددة للمستخدم
4. إضافة نظام تقارير متقدم
5. دعم تصدير البيانات بتنسيقات مختلفة

✓ ملاحظة: تم تطبيق الأنماط في الكلاسات التالية:

- | | |
|--------------|--------------------------------------|
| 1. Main.java | - الواجهة الرئيسية |
| 2. Singleton | - LibraryManager.java (النمط الفردي) |
| 3. Composite | - Book.java (الكتاب) |
| 4. Composite | - Section.java (القسم) |
| 5. Composite | - LibraryItem.java (الواجهة) |
| 6. Command | - Command.java (واجهة الأمر) |

- 7. Command - AddBookCommand.java (أمر إضافة كتاب)
- 8. Command - BorrowBookCommand.java (أمر استعارة)
- 9. Command - ReturnBookCommand.java (أمر إرجاع)
- 10. Command - DeleteBookCommand.java (أمر حذف كتاب)
- 11. Command - DeleteSectionCommand.java (أمر حذف قسم)
- 12. Command - Invoker.java (منفذ الأوامر)
- 13. LoginSystem.java - نظام تسجيل الدخول
- 14. User.java - بيانات المستخدم